

Interpretable rule splits, a STACD provenance proposal, per-fortnight water, and safe defaults

Salil Gujar

M.Tech CSE Entry No. 2025MCS2106

July 2026

Where we are, and what week 9 tackled

Done so far

- A 4-class base map over India at 10 m with a living hierarchy: split or add a class, mark a few example polygons, and retrain that node on the fly.
- A git-backed zoo of model and dataset cards, with save and resume, merge, base and year selection, a GeoTIFF export, and site testing on acacia, tea, and mining.

This week (a chosen subset)

- Split a class by an interpretable rule on indices, not only by a trained model.
- Read the STACD paper, and propose how a provenance record for our outputs would look.
- Answer water seasonality on raw Sentinel, one fortnight at a time.
- Add safe defaults: a bounding-box size cap and a training-time benchmark.
- Link the valid model families to the inference data the user picks.

Splitting a class by a rule, not a model

- Until now, dividing a class meant marking data and training a classifier. Sometimes the boundary is simply a threshold on a known index, for example dense against sparse vegetation by annual NDVI.
- A rule split lets the user write that directly: if the expression holds the pixel takes one class, otherwise it takes the default. No data to mark, no training.
- The indices come from a fixed registry, so the expressions stay interpretable and safe. We check every expression against a whitelist of variables and operators before it reaches Earth Engine.
- Because the indices are computed in Earth Engine, a rule split renders as the same crisp 10 m tile map as a linear model. It is stored on the node, travels with the saved project, and gets a card in the zoo like any other model.

The rule variable registry

The indices a rule can use, all computed live from Sentinel and SRTM over the chosen area and year.

Group	Variables
Vegetation	ndvi_annual, ndvi_kharif, ndvi_rabi
Water	ndwi, mndwi
Built and soil	ndbi, bsi
Terrain (SRTM)	slope, elevation

STACD: what I understood from the paper

This is my reading of the STACD paper, presented so we can confirm it together.

- STAC describes an individual geospatial asset well, but its lineage is shallow: a `derived_from` field that names a parent without saying which algorithm, version, or parameters produced it.
- STACD extends STAC with a directed acyclic graph and five classes: a DAG, a Dataset Type and an Algorithm Type as the node kinds, an Algorithm Instance that records a specific version and its code, and a Dataset Instance that extends a STAC Item with the producing algorithm and its inputs.
- The payoff is provenance and selective recomputation: if a model or an input dataset changes, you can see exactly which downstream outputs need to be rebuilt.
- The paper gives a reference implementation on Apache Airflow.

How our output could map onto STACD

Something like this could work for us. A classified output would carry two things.

- A stack specification: a STAC Item for the LULC raster, with the bounding box, the geometry, the class legend with colours, and asset links to the GeoTIFF and tile endpoints.
- A STACD specification: the DAG plus the dataset and algorithm nodes, and one algorithm instance for every live model, rule split, and merge, each pointing at where its artifact lives in the zoo.
- The input set: the class hierarchy, the ordered operations that built it, and the artifact pointers, carried as the inputs of the producing algorithm. This is the JSON we already keep, which would sit inside the STACD record as the input set that produced the output.
- The appeal is that it reuses the cards we already have, so it needs no new source of truth, and it stays a cheap metadata step with no extra Earth Engine work.

Water seasonality, one fortnight at a time

- The annual embedding is a whole-year signal, so it cannot say which pixels held water on a given fortnight. For that we go back to raw Sentinel: Sentinel-1 radar and Sentinel-2 optical, composited around a target date, with the standard water indices.
- We train a linear water against non-water model offline on the shared seasonal-water polygons, sampled at each polygon's own date, then replay it as Earth-Engine band math for any chosen fortnight. This keeps it interactive, the same trick the base map uses, and avoids the memory limit that pushed the earlier flood pipeline to batch export.
- Early results are good enough to work with for now, and we can tighten them as we add more polygons and dates.

A size cap on the drawn area

- A user can draw any rectangle, and a large one can quietly turn into a very slow request or a large download. We now check the area before running.
- The three paths cost differently, so the caps differ: the Earth-Engine tile map is generous because tiles render on demand, the GeoTIFF export is smaller because that download is size limited, and the Tesseract path is capped by tile count because each tile is a large local download.
- The caps are plain environment settings, so a server administrator can tune them for the hardware. The interface reports the area and refuses to run past the hard limit, with a readable reason.

How long does training take

A benchmark separates the two costs so an administrator can size the server. Sampling embeddings over Earth Engine is network bound and usually dominant; the linear fit is cheap.

Random Forest is a local reference only.

Area (km ²)	Points at 10%	Sample (s)	Total, linear (s)
1	1,000	8.1	8.1
4	4,000	16.1	16.2
25	25,000	72.7	72.9
100	100,000	274.5	275.7

Measured on a Windows laptop. The numbers feed a saved profile and a live estimate endpoint, and they are the reference for setting the area caps above.

Linking models to the inference data

- When a user trains, the valid model families now follow the data they picked. Earth Engine can only run linear models, because only a linear model replays as band math, so the Alpha Earth list is linear only.
- A Tessera run is a local computation, so it is not limited to linear. It adds Random Forest, and XGBoost when installed, and reserves a slot for an object-detection family we have not built yet.
- The training panel reads this list, so the user is only offered models that fit their data, and a non-linear model on Alpha Earth is refused with a clear reason.
- This is the linkage advised earlier: the dataset a user works with determines the models available for it, and the set is easy to extend.

Improving the acacia split

Acacia against non-acacia is our hardest split, a species distinction between similar trees. The options we have actually measured on it:

- More labelled crowns across sites and years. Data spread is the proven lever: pooling several years moved this split from 0.635 to 0.745 on years the model had never seen.
- Tessera as the feature source, tested at about 0.73 on the same split. With the linkage above it can now also carry a non-linear model such as Random Forest, which the 64-dimensional Alpha Earth path cannot.
- The automatic bake-off, already run on this split, where Ridge came out ahead at 0.731.

Where the product stands

- The full loop works: paint a map, grow a scheme with splits, rule splits, adds and merges, retrain, browse and publish the zoo, save and resume a project, and export a GeoTIFF. This is a demo-ready product.
- What stands between a demo and a hosted service is mostly packaging: a service-account key for headless Earth Engine, a container, pinned dependencies, and a basic access limit so the compute is not open to anyone. The code is modular, so these are additions, not rewrites.
- The water model is new and is only lightly wired into the interface so far.

Thank you